

UNIVERSIDAD DEL VALLE DE GUATEMALA
Facultad de Ingeniería



Actividad 9

Entry points de Python en el pipeline de scikit-learn

Diego Linares

Andy Fuentes

Christian Echeverria

Diederich Solis

Machine Learning Engineering

Octubre de 2026

Repositorio

<https://github.com/DiegoLinares11/Actividad-9-MLOPS>

Contiene el paquete del pipeline (`src/act9_pipeline/`), un plugin de ejemplo en un paquete aparte (`plugins/act9-modelo-knn/`), el `Makefile`, el workflow de GitHub Actions (`.github/workflows/`), las pruebas (`tests/`), un notebook que recorre los entry points desde Python (`notebooks/`), el paquete construido (`dist/`) y este reporte (`reporte/`).

Resumen

La actividad pide investigar los entry points de Python, adaptar el pipeline de scikit-learn para usarlos y responder tres preguntas. Partimos del pipeline de la Actividad 3 (el dataset de la Champions League del Ejercicio 1, con calibración de hiperparámetros), que ya tenía un único comando, `act3-demo`, que hacía todo de una vez. Lo reorganizamos alrededor de los entry points:

1. **Leímos la especificación de PyPA** y la documentación de uv, Poetry y conda-build sobre cómo declarar y ejecutar entry points.
2. **Un comando por etapa del pipeline** (`act9-datos`, `act9-entrenar`, `act9-evaluar`, `act9-predecir`), más un comando con subcomandos (`act9`) y uno de evidencia (`act9-demo`). Todos son `console_scripts`.
3. **Las familias de modelos como plugins**. Se registran en un grupo propio, `act9_pipeline.modelos`, y `act9-entrenar` las descubre al arrancar. Un paquete externo agregó una familia nueva (KNN) sin tocar el código del pipeline.
4. **Un Makefile** que encadena los comandos y un **workflow de GitHub Actions** que solo llama al Makefile.
5. **Probamos el mismo proyecto con pip, conda, uv y Poetry**, en Windows.

Resultados principales:

Resultado	Evidencia
Reorganizar el código no cambió el modelo	Con las familias en el orden de la Actividad 3: CV de 0.619, LogisticRegression, test 0.690 / 0.631, igual que allá
Un paquete externo agregó un modelo sin tocar el pipeline	Al instalar <code>act9-modelo-knn</code> , <code>act9_modelos</code> pasó de 4 a 5 familias
El mismo <code>pyproject.toml</code> funcionó con los cuatro gestores	<code>pip</code> , <code>conda</code> , <code>uv</code> y <code>Poetry</code> dieron el mismo resultado: CV 0.652, test 0.655 / 0.604
El código de salida detiene el pipeline	Con <code>UMBRAL=0.90</code> , <code>act9-evaluar</code> devolvió 1 y <code>make</code> se detuvo antes de <code>predecir</code>
<code>make</code> solo repite lo que cambió	La segunda vez: <code>Nothing to be done for 'pipeline'</code>

1. Investigación: qué es un entry point

Según la especificación de la Python Packaging Authority (s.f.-a), los entry points son un mecanismo para que un paquete instalado anuncie componentes que otro código puede usar. Los usos más comunes son dos: crear comandos de terminal y permitir que una aplicación cargue plugins que vienen en otros paquetes. La especificación formalizó en 2017 una funcionalidad que venía de setuptools.

Las tres partes de un entry point

Parte	Qué indica	Ejemplo del pipeline
Grupo	Qué tipo de objeto ofrece. Los grupos nuevos deberían empezar con el nombre del proyecto que los consume, seguido de un punto	<code>console_scripts</code> , <code>act9_pipeline.modelos</code>
Nombre	Cómo se llama dentro del grupo. Se recomienda usar solo letras, números, guiones bajos, puntos y guiones	<code>act9-entrenar</code> , <code>svc</code>
Referencia	Dónde está el objeto, como <code>modulo</code> o <code>modulo:objeto.atributo</code> . Para cargarlo se importa el módulo y se busca el objeto	<code>act9_pipeline.cli.entrenar:main</code>
Extras	Dependencias opcionales que necesita el entry point. Están obsoletos : las herramientas nuevas no tienen que soportarlos	(no se usan)

Dónde quedan guardados

Al instalar un paquete, el instalador escribe sus entry points en el archivo `entry_points.txt`, dentro de la carpeta `.dist-info` del paquete. Tiene formato INI: cada sección es un grupo y cada línea es `nombre = referencia`. La especificación pide UTF-8 y que los nombres distingan mayúsculas de minúsculas. Este es el que trae el wheel de nuestro paquete:

```
[act9_pipeline.modelos]
hist_gradient_boosting = act9_pipeline.modelos:hist_gradient_boosting
random_forest = act9_pipeline.modelos:random_forest
regresion_logistica = act9_pipeline.modelos:regresion_logistica
svc = act9_pipeline.modelos:svc
```

```
[console_scripts]
act9 = act9_pipeline.cli:main
act9-datos = act9_pipeline.cli.datos:main
act9-demo = act9_pipeline.cli.demo:main
```

```
act9-entrenar = act9_pipeline.cli.entrenar:main
act9-evaluar = act9_pipeline.cli.evaluar:main
act9-predecir = act9_pipeline.cli.predecir:main
```

El wheel no trae ningún ejecutable, solo este archivo de texto. Los ejecutables se crean al instalar, según el sistema operativo, y por eso el mismo wheel sirve en Windows y en Linux.

Los grupos especiales: `console_scripts` y `gui_scripts`

En estos dos grupos la referencia debe apuntar a una **función que se llama sin argumentos**. Por cada línea, el instalador crea un ejecutable en la carpeta de scripts del ambiente (`Scripts\` en Windows, `bin/` en Linux) que llama a esa función. Lo que la función devuelve es el **código de salida** del proceso, y si devuelve `None` se toma como 0.

La diferencia entre los dos grupos solo existe en Windows: un `console_scripts` se conecta a una consola y puede leer y escribir en ella, y un `gui_scripts` se abre sin consola. En los demás sistemas son iguales. La especificación también aclara que los instaladores **no** son responsables de agregar la carpeta de scripts al `PATH`.

Grupos propios: plugins

Cualquier otro grupo no crea ejecutables: solo deja el registro en la metadata. Una aplicación lee los grupos que le interesan con `importlib.metadata`, de la biblioteca estándar desde Python 3.8, y carga cada objeto con `load()`. La guía de PyPA sobre plugins lo presenta como el método basado en metadata, junto a otros dos: los prefijos de nombre (como `flask_`) y los *namespace packages* (Python Packaging Authority, s.f.-b).

```
from importlib.metadata import entry_points

for ep in entry_points(group="act9_pipeline.modelos"):
    fabrica = ep.load()      # importa el modulo y toma la funcion
    print(ep.name, fabrica())
```

Así funcionan los plugins de `pytest` (grupo `pytest11`), los comandos de `Flask` (`flask.commands`) y los backends de `MLflow` (`mlflow.tracking_store`).

Cómo se declaran en `pyproject.toml`

Tabla	Grupo que genera
<code>[project.scripts]</code>	<code>console_scripts</code>
<code>[project.gui-scripts]</code>	<code>gui_scripts</code>
<code>[project.entry-points."grupo"]</code>	Cualquier otro grupo, por ejemplo uno de plugins

Antes de `pyproject.toml` se declaraban con el argumento `entry_points` de `setup()` en `setup.py`, y se leían con `pkg_resources`, que hoy está reemplazado por `importlib.metadata`.

2. Adaptación del pipeline

Antes y después

En la Actividad 3 el paquete tenía un solo entry point, `act3-demo`, que cargaba los datos, corría tres búsquedas de hiperparámetros, evaluaba y guardaba el diagrama, todo en una función. Servía como evidencia, pero no para producción: no se podía reentrenar sin repetir todo, ni predecir con un modelo ya entrenado. Las familias de modelos, además, estaban escritas a mano en `tuning.py`.

Ahora cada etapa de CRISP-DM es un comando, y las etapas se comunican por archivos:

Comando	Etapas	Qué hace	Escribe
<code>act9-datos</code>	Data Preparation	Valida las columnas, limpia (151 → 144 partidos) y separa train/test estratificado	<code>train.csv</code> , <code>test.csv</code>
<code>act9-entrenar</code>	Modeling	<code>RandomizedSearchCV</code> sobre las familias descubiertas como plugins	<code>modelo.joblib</code> , <code>metadatos.json</code>
<code>act9-evaluar</code>	Evaluation	Métricas, baseline y matriz de confusión. Devuelve 1 si <code>f1_macro</code> queda bajo <code>--umbral</code>	<code>metricas.json</code>
<code>act9-predecir</code>	Deployment	Predice partidos nuevos (y da probabilidades si el modelo las tiene)	<code>predicciones.csv</code>
<code>act9 <etapa></code>	—	Las mismas funciones como subcomandos, más <code>info</code> y <code>modelos</code>	—
<code>act9-demo</code>	—	Equipo, ambiente, entry points y las cuatro etapas en una carpeta temporal	—

Los módulos `data.py`, `transformers.py` y `pipeline.py` son los de la Actividad 3 sin cambios. Lo nuevo está en `cli/` (una función `main` por comando), `plugins.py` (el descubrimiento) y `modelos.py` (las cuatro familias propias).

En `pyproject.toml` quedaron así:

```
[project.scripts]
act9 = "act9_pipeline.cli:main"
act9-demo = "act9_pipeline.cli.demo:main"
act9-datos = "act9_pipeline.cli.datos:main"
act9-entrenar = "act9_pipeline.cli.entrenar:main"
act9-evaluar = "act9_pipeline.cli.evaluar:main"
act9-predecir = "act9_pipeline.cli.predecir:main"

[project.entry-points."act9_pipeline.modelos"]
random_forest = "act9_pipeline.modelos:random_forest"
regresion_logistica = "act9_pipeline.modelos:regresion_logistica"
```

```
svc = "act9_pipeline.modelos:svc"
hist_gradient_boosting = "act9_pipeline.modelos:hist_gradient_boosting"
```

Las dos formas de exponer el pipeline, un comando por etapa y uno con subcomandos, apuntan a las mismas funciones. La primera es más cómoda para un Makefile, un Dockerfile o un orquestador. La segunda, al estilo de `git`, se descubre más fácil con `act9 --help`.

Qué crea el instalador

Después de `pip install -e .`, la carpeta `.venv\Scripts` tiene un `.exe` por comando. Abrimos uno: es un lanzador de 108 KB que lleva dentro un ZIP con este `__main__.py`:

```
import sys
from act9_pipeline.cli.entrenar import main
if __name__ == '__main__':
    sys.argv[0] = sys.argv[0].removesuffix('.exe')
    sys.exit(main())
```

Es exactamente lo que dice la especificación: importa la función, la llama sin argumentos y usa lo que devuelve como código de salida. Por eso cada `main()` del paquete devuelve 0 (todo bien), 1 (falló una validación o la compuerta de calidad) o 2 (error de uso). `uv` crea un lanzador equivalente de 47 KB, y Poetry, al instalar el proyecto en modo editable, crea un script de Python más un `.cmd` que lo ejecuta.

Las familias de modelos como plugins

Cada familia es una función sin argumentos que devuelve un diccionario con el estimador, su espacio de hiperparámetros y una descripción. Este es el contrato del grupo `act9_pipeline.modelos`. Al arrancar, `act9-entrenar` hace lo siguiente:

1. Lee el grupo con `entry_points(group="act9_pipeline.modelos")`, es decir, de **todos** los paquetes instalados.
2. Los ordena por nombre, para que el espacio de búsqueda no dependa del orden en que el sistema de archivos devuelve los paquetes.
3. Carga y valida cada uno. Si uno falla (no se puede importar, le faltan llaves o pide un hiperparámetro que el estimador no tiene), avisa y lo omite sin detener el entrenamiento.
4. Arma un `RandomizedSearchCV` con un espacio por familia, agregando el prefijo `modelo__` y los hiperparámetros comunes del preprocesamiento.

Para probarlo de verdad, escribimos el plugin en **otro paquete**: `plugins/act9-modelo-knn`. Tiene su propio `pyproject.toml`, no tiene comandos y no importa nada de `act9_pipeline`. Solo se registra en el grupo:

```
[project.entry-points."act9_pipeline.modelos"]
knn = "act9_modelo_knn:knn"
```

Después de `pip install -e plugins/act9-modelo-knn`, `act9 modelos` pasó de 4 a 5 familias y la nueva aparece con su paquete de origen:

knn

KNeighborsClassifier (plugin externo act9-modelo-knn)

referencia: act9_modelo_knn:knn

paquete: act9-modelo-knn 0.1.0

calibra: n_neighbors, weights, p

Resultados

Todas las semillas están fijas, así que los resultados se repiten en cualquier computadora con las mismas familias instaladas.

Familias en la búsqueda	Mejor CV	Ganador	Test (accuracy / f1_macro)
Orden de la Actividad 3 (RF, LR, SVC, HGB)	0.619	LogisticRegression	0.690 / 0.631
todas, en orden alfabético (por defecto)	0.652	SVC	0.655 / 0.604
todas con el plugin knn (5 familias)	0.622	SVC	0.655 / 0.604
Solo knn	0.559	KNeighbors Classifier	0.621 / 0.534
Baseline (clase mayoritaria)	—	—	0.483 / 0.217

La primera fila reproduce exactamente la Actividad 3, lo que confirma que reorganizar el código no cambió el modelo. Las demás dejan dos lecciones:

- **Instalar un plugin cambia el resultado.** RandomizedSearchCV elige al azar uno de los espacios de la lista antes de muestrear, así que agregar una familia, o cambiar el orden, cambia qué combinaciones se prueban. Con 5 familias el mejor CV bajó de 0.652 a 0.622 aunque ganó la misma familia. En producción hay que fijar `--familias` explícitamente.
- **Un mejor CV no garantiza un mejor test.** El orden alfabético ganó en CV pero perdió en test. Con 29 partidos de prueba, la diferencia es de un solo acierto (19 contra 20).

El paquete tiene 14 pruebas con pytest que revisan que los comandos y las familias estén registrados en la metadata, que un plugin externo se descubra y entrene, que un plugin roto se omita, que la compuerta devuelva 1, que el código de salida llegue al sistema a través del ejecutable y que la calibración siga dando 0.619 con el orden de la Actividad 3.

3. ¿Qué usos tendría un entry point en ambientes de producción?

1. **Una interfaz estable para ejecutar el código.** El Dockerfile, el cron, Airflow o el job de Kubernetes llaman a `act9-entrenar` y no a la ruta de un archivo. Si el código se reorganiza, solo cambia una línea del `pyproject.toml`. En la Actividad 4, el contenedor del entrenador copiaba el script a mano y hacía `CMD ["python",`

"entrenar.py"]. Con entry points bastaría instalar el wheel, como en este Dockerfile de ejemplo:

```
FROM python:3.12-slim
COPY dist/act9_pipeline-0.1.0-py3-none-any.whl /tmp/
RUN pip install --no-cache-dir /tmp/act9_pipeline-*.whl
ENTRYPOINT ["act9"]
CMD ["entrenar"]          # docker run imagen predecir --entrada ...
```

2. **Un comando por etapa.** Cada paso se programa, se reintenta y se escala por separado: un DAG de Airflow con una tarea por comando, o un *CronJob* que solo corre `act9-predecir` después de cada jornada, sin volver a entrenar.
3. **El código de salida como contrato.** Como la función devuelve el código de salida, `act9-evaluar --umbral 0.5` funciona como compuerta de calidad: si el modelo no alcanza la métrica, termina con 1 y `make`, GitHub Actions o el orquestador se detienen sin leer la salida.
4. **Extender sin redespregar.** Con un grupo de plugins, un equipo agrega un modelo, un conector de datos o una métrica publicando otro paquete, sin tocar ni volver a probar el paquete principal.
5. **Trazabilidad.** El comando queda amarrado a la versión instalada del paquete, no a un archivo suelto en el servidor. Nuestro `metadatos.json` guarda qué comando y qué versión entrenaron el modelo y de qué paquete vino la familia ganadora.

Una advertencia: los `gui_scripts` casi no tienen uso en producción, porque los servidores y los contenedores no tienen interfaz gráfica.

4. ¿Se podría utilizar un entry point con un manejador de paquetes?

Sí, con todos. Los entry points son parte del estándar de empaquetado (`[project.scripts]` y `[project.entry-points]` en `pyproject.toml`, `entry_points.txt` en la metadata), así que cualquier instalador que lo siga los entiende. Lo comprobamos con el mismo `pyproject.toml` en Windows, y la tabla resume lo que observamos con cada gestor.

Con los cuatro, `make pipeline` dio CV de 0.652 y test de 0.655 / 0.604. Algunas diferencias que vale la pena conocer:

- **uv** necesita una sección `[build-system]` para instalar los entry points (Astral, s.f.). Sin ella trata el proyecto como una aplicación que no se empaqueta.
- **Poetry 2** recomienda `[project.scripts]` y marca como obsoleta su tabla antigua `[tool.poetry.scripts]` (Poetry, s.f.). Nos dio dos sorpresas. Pidió que existiera el `README.md` declarado en `pyproject.toml`, aunque `pip` y `uv` ya habían instalado el proyecto sin él. Además, como ya había un `.venv` en la carpeta, lo usó y reemplazó los `.exe` de `pip` por los suyos. Conviene un ambiente por herramienta.
- **conda** no lee `pyproject.toml`. La forma práctica es la sección `pip:` de `environment.yml`, y así `conda` instala las dependencias pesadas y `pip` el paquete. Para publicar un paquete de `conda`, la receta de `conda-build` tiene su propia

Gestor	Cómo se instala	Cómo se ejecuta	Lo que observamos
pip	<code>pip install -e .</code> en un venv	Activar el ambiente y <code>act9-entrenar</code>	Lanzadores <code>.exe</code> en <code>.venv\Scripts</code> ; Python 3.14.2
conda	<code>conda env create</code> <code>-f environment.yml</code> (con <code>pip: -e .</code>)	Activar el ambiente, o <code>conda run -n</code> <code>act9-mlops</code> <code>act9-entrenar</code>	Los <code>.exe</code> quedan en <code>envs\act9-mlops\</code> <code>Scripts</code> ; Python 3.12.14
uv	Automático al correr	<code>uv run</code> <code>act9-entrenar</code> ; <code>uvx</code> <code>--from . act9 info</code>	Crea y sincroniza el ambiente solo; <code>uvx</code> usa uno temporal
Poetry 2.5	<code>poetry install</code>	<code>poetry run</code> <code>act9-entrenar</code>	Lee <code>[project.scripts]</code> ; crea un script y un <code>.cmd</code>

sección, que genera los mismos ejecutables (conda-build, s.f.):

```
build:
  entry_points:
    - act9-entrenar = act9_pipeline.cli.entrenar:main
```

- **pipx** y **uv tool install** instalan los comandos de un paquete en un ambiente aislado, para usarlos como herramientas del sistema. Probamos el equivalente temporal, **uvx**.
- En los cuatro casos, si la carpeta de scripts del ambiente no está en el **PATH**, los comandos no existen aunque el paquete esté instalado. Nos pasó al instalar el wheel en un venv sin activarlo: **act9-demo** respondió **command not found** y, llamado con su ruta completa, reportó **NO ESTA EN EL PATH** para cada comando. El plan B es **python -m act9_pipeline**, que solo necesita que el paquete se pueda importar.

5. ¿Qué relación podría tener un entry point con un Makefile?

Un Makefile es un archivo de instrucciones para automatizar tareas: cada *objetivo* tiene sus *dependencias* y una *receta* con los comandos que lo producen, y el programa **make** las ejecuta (Awan, 2024). La relación con los entry points es de capas:

- **El entry point define qué se ejecuta:** la interfaz de cada etapa, dentro del paquete.
- **El Makefile define cuándo y en qué orden:** las dependencias entre etapas, fuera del paquete.
- **GitHub Actions llama al Makefile**, como propone el tutorial de DataCamp. Así el mismo **make pipeline** corre en la laptop y en CI.

En nuestro Makefile cada objetivo es el archivo que produce una etapa y cada receta llama a un entry point:

```
$(TRAIN) $(TEST) &: $(CSV_CRUDO)
$(RUN) act9-datos --salida $(DIR_DATOS)
```

```
$(MODELO): $(TRAIN)
$(RUN) act9-entrenar --entrada $(TRAIN) --salida modelos \
--familias $(FAMILIAS) --n-iter $(N_ITER)

$(METRICAS): $(MODELO) $(TEST)
$(RUN) act9-evaluar --modelo $(MODELO) --entrada $(TEST) \
--salida reportes --umbral $(UMBRAL)

$(PREDICCIONES): $(MODELO) $(TEST) $(METRICAS)
$(RUN) act9-predecir --modelo $(MODELO) --entrada $(TEST) \
--salida $(PREDICCIONES)
```

Cada uno aporta lo que al otro le falta:

El entry point le da al Makefile	El Makefile le da al entry point
Recetas cortas que no dependen de rutas internas del código	El orden entre etapas
Códigos de salida confiables para saber si debe detenerse	Ejecución incremental: solo repite las etapas cuyas entradas cambiaron
La misma interfaz con cualquier gestor de paquetes (variable RUN)	Variables que se cambian al vuelo: make entrenar N_ITER=20 UMBRAL=0.6

El objetivo `make install` es el punto de unión: es el que corre `pip install -e .` y crea los comandos que usan los demás objetivos.

Comprobamos el comportamiento en Windows con GNU Make 4.4.1, que instalamos desde conda-forge dentro del ambiente de conda. La tabla muestra qué hizo make en cada situación, y esta fue la salida cuando el modelo no pasó la compuerta:

Situación	Qué hizo make
make pipeline desde cero	Corrió las cuatro etapas
make pipeline otra vez	Nothing to be done for 'pipeline'
El modelo quedó más nuevo que las métricas	Solo repitió act9-evaluar y act9-predecir
make pipeline UMBRAL=0.90	act9-evaluar devolvió 1, make se detuvo con Error 1, borró <code>metricas.json</code> y no corrió la predicción
make pipeline RUN="uv run" y RUN="poetry run"	El mismo pipeline y el mismo resultado con otro gestor

```
[act9-evaluar] FALLA: f1_macro=0.604 < umbral 0.900 -> codigo de salida 1
make: *** [Makefile:84: reportes/metricas.json] Error 1
make: *** Deleting file 'reportes/metricas.json'
```

Que make borre el archivo se debe a `.DELETE_ON_ERROR`. Sin esa línea, una evaluación fallida dejaría un `metricas.json` con fecha nueva y la siguiente corrida creería que ya no

hay nada que hacer. También declaramos `.PHONY` para los objetivos que no son archivos (`datos`, `test`, `clean`, ...), algo que el tutorial no muestra.

El workflow de GitHub Actions tiene dos jobs, uno con pip y otro con uv, y ninguno repite comandos del pipeline:

```
- run: make install-dev
- run: make test
- run: make pipeline UMBRAL=0.50
- run: make pipeline RUN="uv run"      # job de uv
```

Los dos jobs pasaron en el primer intento, en Ubuntu con Python 3.12 (pip) y con el Python que eligió uv. Entre las pruebas del job de pip está la que exige el 0.619 de la Actividad 3, así que el resultado también se reproduce en Linux.

Conclusiones

1. **Un entry point es una función con nombre.** Abrir el `.exe` que generó pip lo deja claro: importa la función y llama `sys.exit(main())`. Todo lo demás (dónde queda el ejecutable, si es `.exe` o `.cmd`) lo decide el instalador. Por eso el mismo `pyproject.toml` funcionó con pip, conda, uv y Poetry.
2. **Dividir el pipeline en comandos lo acerca a producción.** Con un solo `act3-demo` no se podía predecir sin reentrenar. Con un comando por etapa, cada una se corre, se programa y se reintenta por separado, y las etapas se comunican por archivos que se pueden guardar o revisar.
3. **El código de salida es la parte más útil para MLOps.** Que `act9-evaluar` devuelva 1 cuando el modelo no alcanza el umbral permitió que make, y luego GitHub Actions, detuvieran el pipeline antes de predecir con un modelo malo, sin leer la salida.
4. **Los plugins dan flexibilidad, pero también variabilidad.** Instalar `act9-modelo-knn` agregó una familia sin tocar el pipeline, pero también cambió el resultado de la búsqueda. Lo que hay instalado en el ambiente pasa a ser parte del modelo, así que en producción hay que fijar las familias y las versiones de los paquetes.
5. **El Makefile y el entry point son capas distintas.** El entry point dice qué hace cada etapa, el Makefile cuándo correrla y GitHub Actions dónde. Cada capa queda corta y legible, y la misma orden (`make pipeline`) sirve en Windows, en Linux y en CI.
6. **En nuestra opinión, lo que más enseñó fue lo que falló.** Los comandos no se encontraron al no activar el ambiente, Poetry reemplazó los ejecutables de pip en el mismo `.venv`, y el notebook volvió a arrancar con el Python global como en el Taller 2. En los tres casos el código estaba bien y el problema era el ambiente. Por eso `act9 info` imprime el intérprete, el gestor y la ruta de cada comando.

Referencias

Astral. (s.f.). *Configuring projects: Entry points*. Documentación de uv. <https://docs.astral.sh/uv/concepts/projects/config/>

Awan, A. A. (2024). *GitHub Actions and MakeFile: A hands-on introduction*. DataCamp.
<https://www.datacamp.com/tutorial/makefile-github-actions-tutorial>

conda-build. (s.f.). *Defining metadata (meta.yaml): Python entry points*. <https://docs.conda.io/projects/conda-build/en/stable/resources/define-metadata.html>

Poetry. (s.f.). *The pyproject.toml file*. <https://python-poetry.org/docs/pyproject/>

Python Packaging Authority. (s.f.-a). *Entry points specification*. <https://packaging.python.org/en/latest/specifications/entry-points/>

Python Packaging Authority. (s.f.-b). *Creating and discovering plugins*. <https://packaging.python.org/en/latest/guides/creating-and-discovering-plugins/>